

Sistema de Gestión Integral Plaza Max

Willian Li Liang
Ian Alvarez
Fecha: 15/2/2026

1	INTRODUCCIÓN	3
1.1	Alcance	3
1.2	Objetivos.....	3
1.3	Diagrama CU original	3
1.4	CU Detallados.....	3
1.5	Diagrama de secuencia x CU.....	3
2	DESCRIPCIÓN DEL ENTORNO TECNOLÓGICO	4
2.1	Elementos de la Infraestructura	4
3	DEFINICIÓN DE LA ARQUITECTURA DEL SISTEMA	5
3.1	Estilo Arquitectónico	5
3.2	Descripción de la arquitectura de la tecnología.....	5
3.3	Patrones de Diseño y Buenas Prácticas	5
4	DISEÑO DEL MODELO DE CLASES DEL SISTEMA.....	6
4.1	Diseño de la Lógica de Negocio.....	6
5	MODELO FÍSICO DE DATOS	7
5.1	Modelo Físico de Datos.....	7
6	DEFINICIÓN INTERFAZ DE USUARIO DEL SISTEMA	8
6.1	Especificación de la Interfaz de Pantalla y de Navegación.....	8
6.2	Catálogo de Controles y Elementos de Diseño	8
6.3	Definición de Informes.....	8
7	DEFINICIÓN INTERFAZ DE SERVICIOS DEL SISTEMA	9
7.1	Definición de la Interfaz de Servicios del Sistema.....	9
7.2	Servicios Consumidos por el Sistema	9
8	INFORMACIÓN SOBRE TRAZABILIDAD	11
9	ANEXOS [Opcional].....	12
9.1	Anexo A. Glosario de Acrónimos y Abreviaturas	12

1 INTRODUCCIÓN

1.1 Alcance

El presente proyecto abarca el análisis, diseño y desarrollo del **Sistema de Gestión Integral Plaza Max (MaxLi Manager)**. El sistema está diseñado para centralizar el ciclo operativo completo de la empresa, gestionando el flujo de mercancía desde la adquisición a proveedores, su recepción en almacén, hasta la venta final al cliente, permitiendo el control de stock y la validación financiera en tiempo real.

El alcance funcional incluye:

- **Gestión de Inventario y Almacén:** Control de entradas por compras, registro de salidas por ventas o mermas, gestión de ubicaciones físicas (estantes) y monitoreo automatizado de niveles de existencia mediante "Stock Alarma".
- **Gestión de Compras y Proveedores:** Ciclo completo de adquisición, incluyendo la creación de proveedores, emisión de órdenes de compra y validación de recepción de mercancía.
- **Punto de Venta (POS) y Facturación:** Interfaz para el procesamiento ágil de ventas, cálculo de impuestos, manejo de múltiples formas de pago y descuento automático de inventario.
- **Gestión Financiera Operativa:** Control de Cuentas por Pagar mediante validación de tres vías (Orden vs. Recepción vs. Factura).
- **Gestión de Usuarios:** Control de acceso basado en roles jerárquicos (Administración, Cajero, Almacenero).
- **Reportes Operativos:** Generación de informes de cierre de caja, alertas de reposición y auditoría de movimientos.

El sistema no contempla, en esta fase, módulos de nómina compleja de empleados, contabilidad fiscal externa ni gestión de activos fijos, limitándose estrictamente a la logística comercial, facturación y cuentas por pagar operativas.

1.2 Objetivos

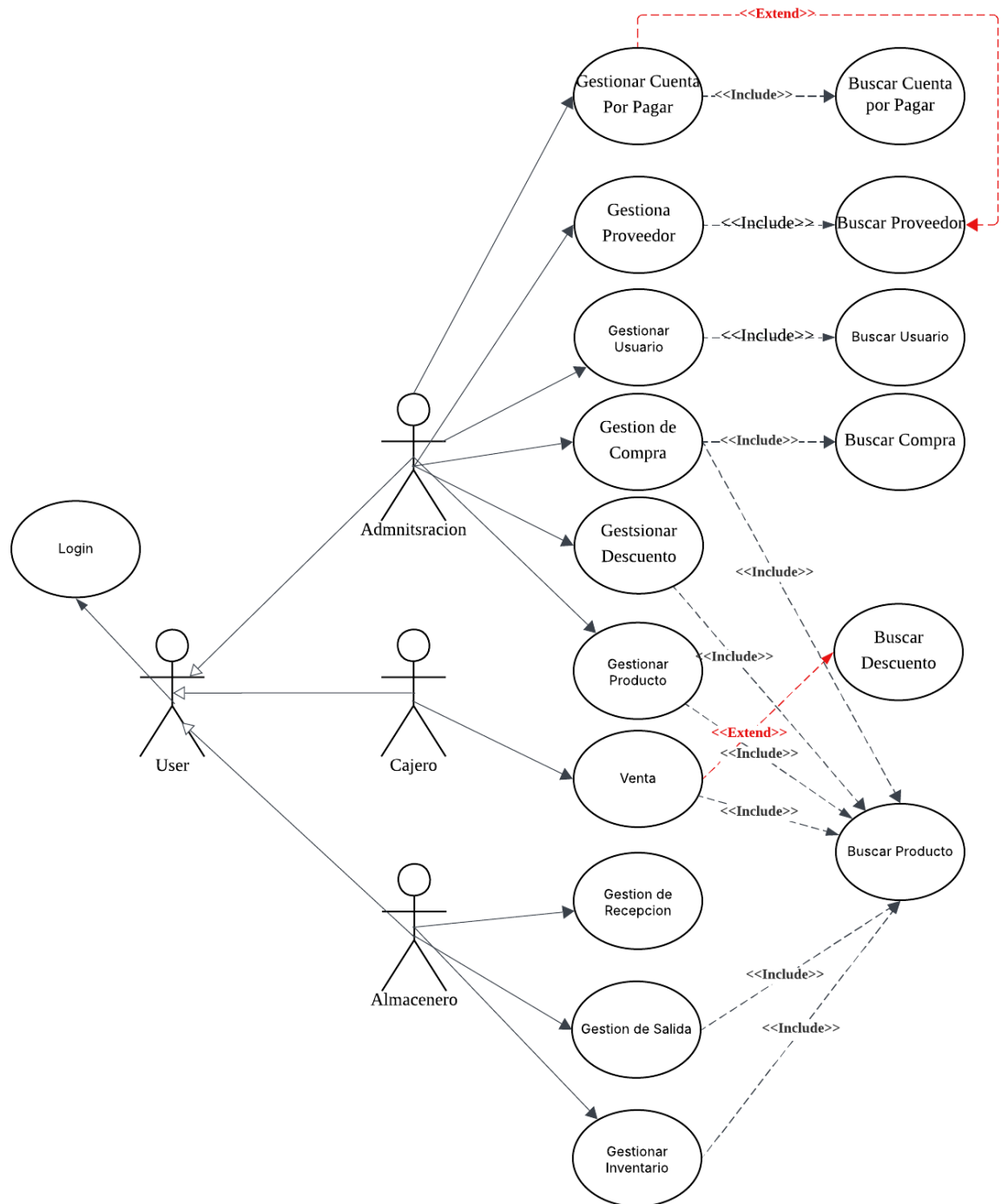
Objetivo General: Desarrollar una solución de software robusta y escalable que unifique los procesos de Compras, Inventario y Ventas de Plaza Max, garantizando la consistencia entre el stock físico y el lógico, y asegurando que cada desembolso financiero esté respaldado por una recepción de mercancía validada.

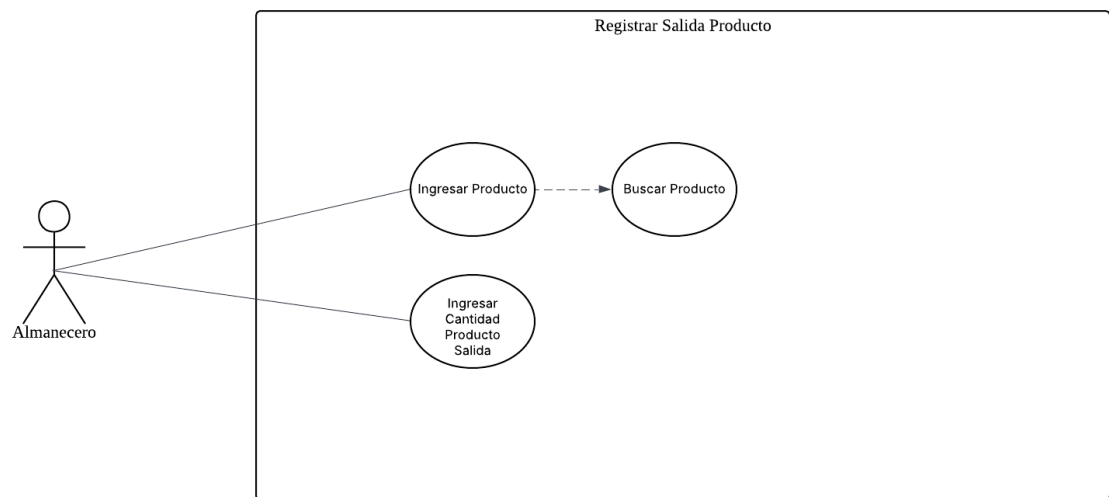
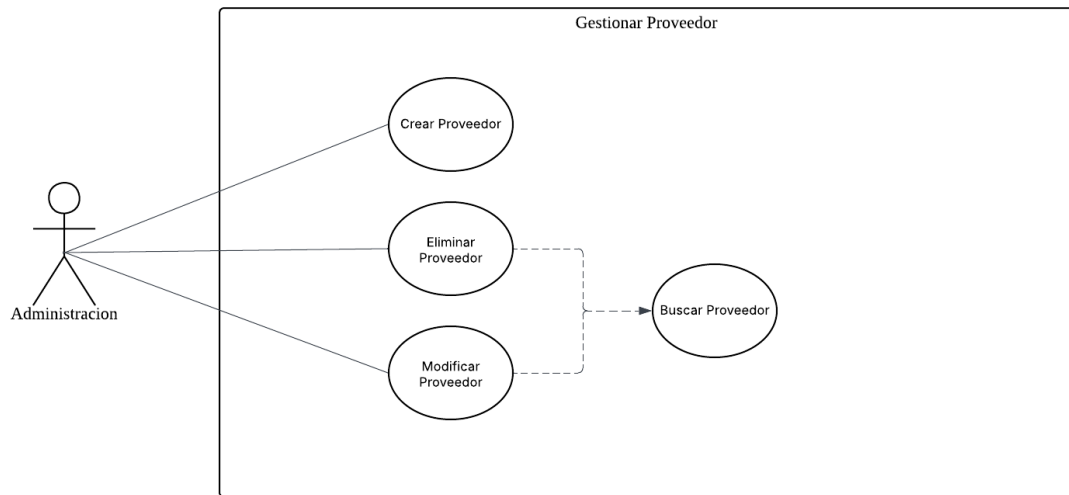
- **Objetivos Específicos:**
 1. **Centralización e Integridad:** Unificar la base de datos para que el módulo de Ventas (Caja) y el de Almacén consulten y actualicen el mismo stock en tiempo real, evitando ventas sin existencias (Stock Negativo).
 2. **Automatización de Reposición:** Implementar el mecanismo de "Stock Alarma" para que el sistema notifique automáticamente la necesidad de compra cuando los niveles de un producto sean críticos, reduciendo la intervención manual reactiva.
 3. **Control Financiero (Validación de 3 Vías):** Restringir sistémicamente la creación de Cuentas por Pagar, impidiendo pagos a proveedores si la mercancía no ha sido marcada como "Recepcionada" por el almacén.
 4. **Trazabilidad:** Implementar un sistema de logs (bitácora) que registre cada movimiento

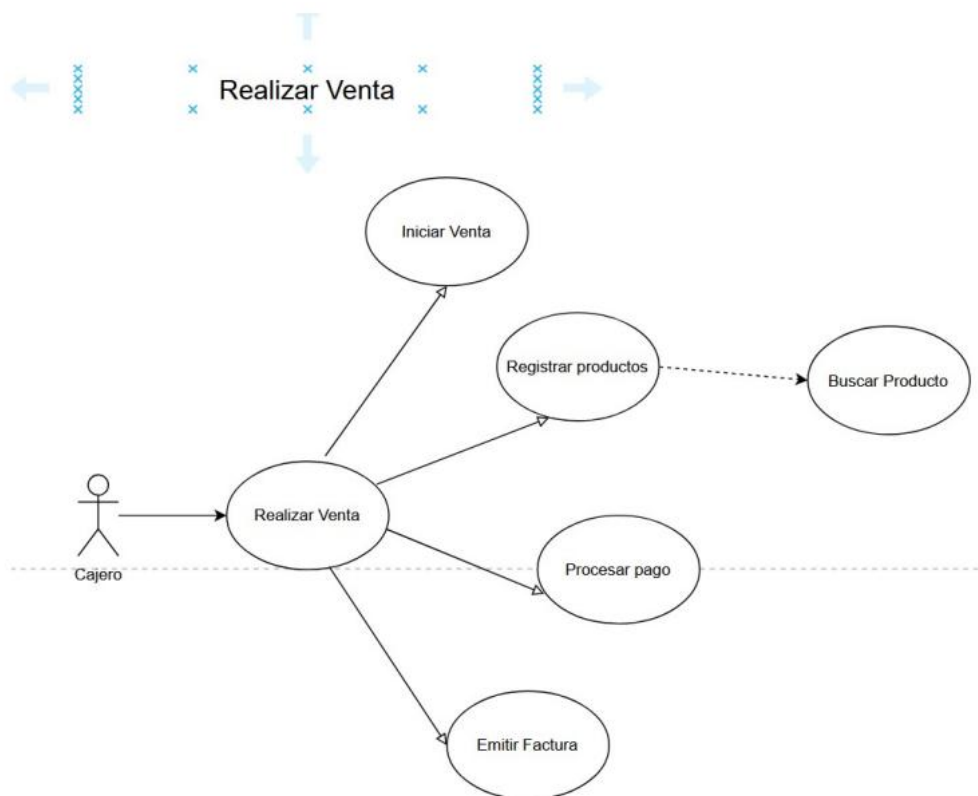
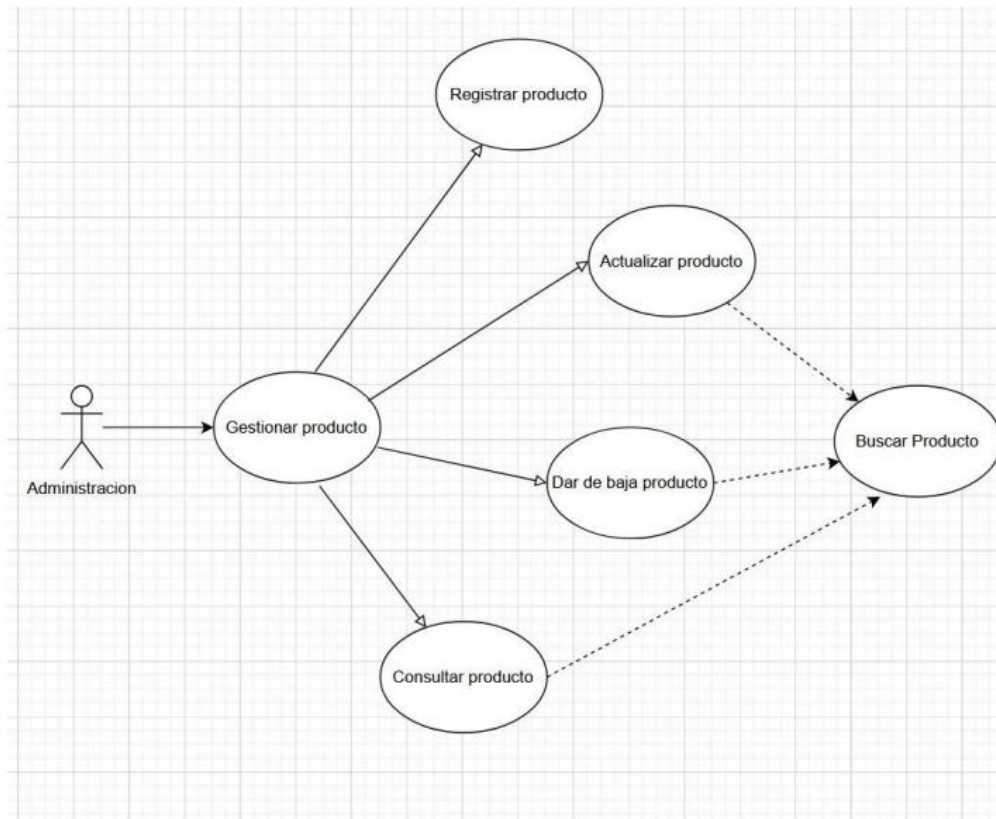
crítico, identificando qué usuario (Cajero/Almacenero/Admin) realizó la acción y en qué momento exacto.

5. **Alta Disponibilidad y Concurrencia:** Diseñar una arquitectura tolerante a fallos que soporte múltiples transacciones de venta simultáneas sin degradar el rendimiento ni corromper los datos del inventario.
6. **Seguridad de Datos:** Asegurar que las eliminaciones de registros maestros (Productos, Proveedores) sean lógicas (cambio de estado a "Inactivo") y no físicas, preservando el historial de operaciones pasadas.

1.3 Diagrama CU original







1.4 CU Detallados

1.5 Diagrama de secuencia x CU

2 DESCRIPCIÓN DEL ENTORNO TECNOLÓGICO

El sistema MaxLi Manager es una aplicación web orientada a la gestión integral de ventas e inventario para tiendas por departamentos. La solución se implementa bajo una arquitectura cliente-servidor donde los usuarios acceden al sistema mediante un navegador web moderno, el frontend es una Single Page Application (SPA) desarrollada con React + Vite, y las solicitudes son procesadas por un servidor de aplicaciones desarrollado en Java utilizando el framework Spring Boot.

El backend está desarrollado con Java LTS y Spring Boot, lo que permite una configuración simplificada, integración con tecnologías modernas y despliegue ágil mediante servidor embebido Apache Tomcat. Para la persistencia de datos se utiliza PostgreSQL, un sistema de gestión de base de datos relacional ampliamente utilizado en entornos empresariales por su robustez, estabilidad y alto rendimiento.

El frontend está desarrollado con React 18 y Vite como bundler, lo que garantiza tiempos de compilación ultrarrápidos, hot module replacement (HMR) en desarrollo y bundles optimizados para producción. La comunicación entre el frontend y el backend se realiza mediante una API REST que expone endpoints JSON, complementada con Axios para las peticiones HTTP desde el cliente.

2.1 Elementos de la Infraestructura

1. Cliente (Capa de Presentación — Frontend)

- Navegador web moderno (Chrome, Edge, Firefox, Safari).
- SPA desarrollada con React 18 + Vite.
- Gestión de estado con React Context API / Zustand.
- Librería de componentes: shadcn/ui o Ant Design para consistencia visual.
- Comunicación con el servidor mediante protocolo HTTP/HTTPS a través de Axios.
- Enrutamiento del lado del cliente con React Router v6.

2. Servidor de Aplicación (Backend)

- Aplicación desarrollada en Java LTS con Spring Boot 3.x.
- Servidor embebido Apache Tomcat ejecutándose en el puerto 8080.
- Exposición de API REST con Spring Web MVC.
- Seguridad con Spring Security + JWT (JSON Web Tokens) para autenticación stateless.
- Componentes principales:
 - Controllers (REST Controllers): reciben y validan solicitudes HTTP.
 - Services (Business Logic): contienen las reglas de negocio.
 - Repositories (Data Access Layer): acceso a datos mediante Spring Data JPA.

3. Base de Datos

- Motor relacional PostgreSQL 15+.
- Almacena información estructurada: Productos, Inventario, Ventas, Compras, Usuarios, Turnos, CajaChica, NCF.
- Migraciones gestionadas con Flyway o Liquibase para control de versiones del esquema.

4. Comunicación

- Protocolo HTTP/HTTPS entre cliente y servidor.
- Formato de intercambio de datos: JSON (REST API).
- CORS configurado en Spring Boot para permitir peticiones desde el dominio del frontend.

5. Entorno de Desarrollo

- IDE recomendado: IntelliJ IDEA (backend) / VS Code (frontend).
- Sistema de control de versiones: Git con repositorio en GitHub/GitLab.
- Build tool backend: Maven.
- Build tool frontend: Vite (npm/yarn).
- Contenerización opcional: Docker + Docker Compose para entorno de desarrollo unificado.

Capa	Tecnología	Versión	Rol
Frontend	React	18.x	Librería de UI para SPA
Frontend	Vite	5.x	Bundler y servidor de desarrollo
Frontend	React Router	6.x	Enrutamiento del lado cliente
Frontend	Axios	1.x	Cliente HTTP para consumo de API REST
Backend	Java	21 LTS	Lenguaje principal del servidor
Backend	Spring Boot	3.x	Framework de aplicación empresarial
Backend	Spring Security	6.x	Autenticación y autorización (JWT)
Backend	Spring Data JPA	3.x	Capa de acceso a datos (ORM)
Backend	Hibernate	6.x	Implementación JPA / ORM
Backend	Apache Tomcat	10.x	Servidor embebido
Base de Datos	PostgreSQL	15+	Motor relacional de producción
DevOps	Maven	3.x	Gestión de dependencias backend
DevOps	Docker	Latest	Contenerización (opcional)

3 DEFINICIÓN DE LA ARQUITECTURA DEL SISTEMA

3.1 Estilo Arquitectónico

La arquitectura propuesta para el sistema MaxLi Manager se basa en un estilo de arquitectura desacoplada cliente-servidor, donde el frontend (React + Vite) y el backend (Spring Boot) son aplicaciones independientes que se comunican exclusivamente a través de una API REST. Dentro del backend se aplica una arquitectura en capas (Layered Architecture) con implementación del patrón MVC (Modelo-Vista-Controlador) adaptado al paradigma REST.

Se adopta este estilo por las siguientes razones:

- Separación total de responsabilidades: el frontend y el backend pueden evolucionar de forma independiente.
- Escalabilidad diferenciada: es posible escalar el backend o el frontend de forma independiente según la carga.
- Claridad estructural y facilidad de mantenimiento mediante capas bien definidas.
- Adecuación para sistemas empresariales de gestión con múltiples roles y módulos.
- Posibilidad de migración futura hacia microservicios sin reescribir el frontend.

Capas de la Arquitectura Backend

Capa	Responsabilidad	Componentes
Capa de API (REST Controllers)	Recibir solicitudes HTTP, validar entrada y serializar respuestas JSON	RestController, DTOs, Exception Handlers
Capa de Lógica de Negocio (Services)	Reglas de negocio: validación de stock, cálculo de totales, control de 3 vías	Services, Validators, Business Rules
Capa de Persistencia (Repositories)	Interacción con la base de datos mediante JPA/Hibernate	JpaRepository, Specifications, Queries
Modelo de Dominio (Entities)	Definición de entidades del negocio mapeadas a tablas SQL	Entities, Enums, Value Objects

Capas de la Arquitectura Frontend

Capa	Responsabilidad	Componentes
Capa de Vistas (Pages)	Páginas completas de la aplicación accesibles por ruta	Dashboard, POS, Inventario, Compras, Reportes
Capa de Componentes (UI)	Componentes reutilizables de interfaz de usuario	Tables, Forms, Modals, Cards, Charts
Capa de Estado (State)	Gestión del estado global de la aplicación	Context/Zustand Stores, React Hooks
Capa de Servicios (API Layer)	Centralización de llamadas a la API REST del backend	apiClient.js, endpoints por módulo

3.2 Descripción de la arquitectura de la tecnología

La arquitectura tecnológica del sistema integra dos aplicaciones desacopladas que se comunican mediante HTTP/JSON:

Flujo General de Funcionamiento

1. El usuario accede al sistema desde un navegador web. React Router renderiza la vista correspondiente a la ruta solicitada.
2. La SPA (Single Page Application) realiza peticiones HTTP a la API REST del backend mediante Axios con el token JWT en el header de autorización.
3. Un RestController de Spring Boot recibe la solicitud, valida el token JWT y extrae el usuario autenticado.
4. El Controller delega la lógica al Service correspondiente.
5. El Service ejecuta las reglas de negocio y consulta o actualiza datos mediante los Repositories (Spring Data JPA).
6. Los Repositories interactúan con PostgreSQL mediante Hibernate.
7. El sistema devuelve una respuesta JSON estructurada (DTO) con el código HTTP correspondiente (200, 201, 400, 403, 404, 500).
8. El frontend procesa la respuesta y actualiza el estado de la aplicación, re-renderizando los componentes afectados.

Justificación Tecnológica

- React + Vite proporciona una experiencia de desarrollo moderna con HMR instantáneo y bundles optimizados, resultando en una interfaz de usuario responsiva y de alto rendimiento.
- Spring Boot es el estándar de facto en aplicaciones empresariales Java, con un ecosistema maduro de librerías y soporte a largo plazo.
- La API REST stateless con JWT permite escalabilidad horizontal y es compatible con futuras integraciones (apps móviles, integraciones de terceros).
- PostgreSQL es robusto, confiable y ampliamente utilizado en producción con soporte a transacciones ACID críticas para la integridad del inventario.
- La separación frontend/backend facilita la colaboración en equipos especializados y la evolución independiente de cada capa.

3.3 Patrones de Diseño y Buenas Prácticas

Patrones de Diseño Aplicados

Patrón	Capa	Aplicación en MaxLi Manager
Repository Pattern	Persistencia	Abstracción del acceso a datos mediante interfaces JpaRepository. Permite cambiar el ORM sin afectar la lógica de negocio.
DTO (Data Transfer Object)	API / Servicio	Objetos específicos para cada request/response de la API. Evita exponer las entidades del dominio directamente.

Patrón	Capa	Aplicación en MaxLi Manager
Service Layer	Negocio	Toda la lógica de negocio reside en clases @Service. Los Controllers son delegadores, no contienen lógica.
Factory Method	Negocio	Creación de objetos complejos como Venta, OrdenCompra y sus detalles mediante métodos factory en los Services.
Observer / Event	Negocio	Spring Application Events para notificaciones desacopladas, ej: StockAlarma dispara evento cuando existencia baja del mínimo.
Strategy	Negocio	Estrategias de pago (efectivo, tarjeta, crédito) encapsuladas en implementaciones de una interfaz PagoStrategy.
Singleton	Configuración	Beans de Spring son Singleton por defecto. Garantiza una única instancia de servicios y repositorios.
Facade	API	El RestController actúa como facade, simplificando la interfaz del sistema para el cliente frontend.

Buenas Prácticas Implementadas

- Eliminación lógica (Soft Delete): Los registros maestros (Productos, Proveedores, Usuarios) nunca se eliminan físicamente. Se cambia su campo 'estado' a 'INACTIVO', preservando el historial operativo.
- Validación de 3 Vías: El sistema impide la creación de Cuentas por Pagar si la OrdenCompra no tiene NotaRecepcion asociada con estado 'Recepcionado'.
- Transacciones (@Transactional): Las operaciones que involucran múltiples tablas (ej: Venta + DetalleVenta + Existencia + Movimiento) se ejecutan dentro de una transacción para garantizar atomicidad.
- Manejo global de excepciones: @ControllerAdvice centraliza el manejo de errores, devolviendo respuestas JSON consistentes con códigos HTTP semánticos.
- Paginación y filtrado: Todas las consultas de listado implementan paginación (Pageable) para garantizar el rendimiento con grandes volúmenes de datos.
- Auditoría: Spring Data Auditing registra automáticamente createdAt, updatedAt, createdBy en todas las entidades críticas.
- Variables de entorno: Las configuraciones sensibles (credenciales de BD, clave JWT) se gestionan mediante variables de entorno, nunca en código fuente.
- CORS Policy: Configuración explícita en Spring Security para permitir únicamente el origen del frontend, rechazando peticiones de dominios no autorizados.

El modelo de clases representa la estructura orientada a objetos del sistema MaxLi Manager, definiendo las entidades del dominio, sus atributos, relaciones y cardinalidades. Este modelo es la base para el mapeo objeto-relacional (ORM) mediante Hibernate/JPA y para la generación del esquema de base de datos PostgreSQL.



Entidad	Descripción	Relaciones Principales
Usuario	Representa a los operadores del sistema. Tiene username, email, passwordHash y estado.	Pertenece a un Rol. Asociado a TurnoCaja y CajaChica.
Rol	Define el nivel de acceso: ADMIN, CAJERO, ALMACENERO.	Uno a muchos con Usuario.
Caja	Representa una caja registradora física con su identificador y estado.	Uno a muchos con TurnoCaja.
TurnoCaja	Registro del turno de un cajero en una caja. Almacena fechas, montos y totales.	Pertenece a Caja y Usuario. Contiene movimientosCaja.

Entidad	Descripción	Relaciones Principales
CajaChica	Fondo de caja chica con saldo base y balance actual.	Asociada a Usuario. Tiene movimientosCaja.
movimientoCaja	Registro individual de ingresos/egresos de caja o caja chica.	Pertenece a TurnoCaja o CajaChica.
Producto	Artículo comercializable con código, nombre, precio y costo.	Pertenece a Categoría y Marca. Tiene Existencia, DetalleVenta, DetalleOrdenCompra.
Categoría	Clasificación de productos (ej: Electrónica, Alimentos).	Uno a muchos con Producto.
Marca	Marca comercial del producto.	Uno a muchos con Producto.
Existencia	Stock actual y mínimo de un producto. Vinculado al almacén.	Uno a uno con Producto. Pertenece a Almacen.
Almacen	Ubicación física del inventario.	Uno a muchos con Existencia.
Movimiento	Registro de entradas/salidas de inventario con motivo y fecha.	Tiene DetalleMovimiento. Relacionado con Producto.
DetalleMovimiento	Detalle de cada movimiento de inventario (cantidades antes/después).	Pertenece a Movimiento y Existencia.
Venta	Cabecera de la transacción de venta con totales, impuestos y estado.	Tiene DetalleVenta. Pertenece a Cliente. Puede tener NotaCredito y Devolucion.
DetalleVenta	Línea de producto dentro de una venta con cantidad, precio y descuento.	Pertenece a Venta y Producto.
Cliente	Entidad que realiza compras. Puede tener crédito.	Uno a muchos con Venta. Tiene Ingresos.
Ingreso	Registro de pagos o ingresos crediticios del cliente.	Pertenece a Cliente.
Devolucion	Devolución de una venta completa o parcial.	Pertenece a Venta. Tiene DetalleDevolucion. Genera NotaCredito.
DetalleDevolucion	Línea de producto devuelto con cantidad.	Pertenece a Devolucion y DetalleVenta.
NotaCredito	Documento que formaliza el crédito al cliente por la devolución.	Pertenece a Devolucion. Referencia SecuenciaNCF.
Proveedor	Empresa suministradora de productos.	Uno a muchos con OrdenCompra.
OrdenCompra	Orden de compra emitida a un proveedor.	Tiene DetalleOrdenCompra. Genera NotaRecepcion.
DetalleOrdenCompra	Línea de producto dentro de la	Pertenece a OrdenCompra y

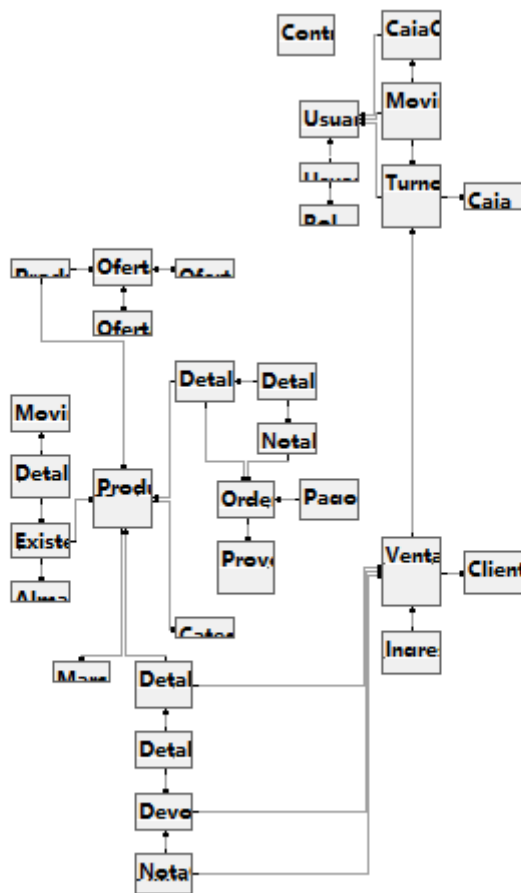
Entidad	Descripción	Relaciones Principales
	orden con cantidad y precio unitario.	Producto.
NotaRecepcion	Confirmación de recepción física de mercancía.	Pertenece a OrdenCompra. Tiene DetalleNotaRecepcion.
DetalleNotaRecepcion	Cantidad efectivamente recibida por producto.	Pertenece a NotaRecepcion.
PagoProveedor	Registro de pagos realizados al proveedor.	Pertenece a OrdenCompra.
Oferta	Promoción aplicable a productos con rango de fechas y estado.	Tiene OfertaCantidad y OfertaDescuento.
OfertaCantidad	Condición de cantidad mínima para activar la oferta.	Pertenece a Oferta.
OfertaDescuento	Porcentaje de descuento aplicado por la oferta.	Pertenece a Oferta.
SecuenciaNCF	Control de secuencias de comprobantes fiscales (NCF).	Referenciada por NotaCredito y Venta.

Reglas de Negocio Críticas Implementadas en la Capa de Servicios

- Stock Alarm: VentaService verifica que Existencia.cantidadActual >= cantidad solicitada antes de procesar la venta. Si la existencia cae por debajo de cantidadMinima, se dispara un evento StockAlarmEvent.
- Validación 3 Vías: CuentaPorPagarService verifica que OrdenCompra.estado = 'RECEPCIONADO' antes de permitir el registro de un PagoProveedor.
- Atomicidad de Venta: VentaService usa @Transactional para garantizar que Venta + DetalleVenta + Existencia (decremento) + Movimiento se persistan en una sola transacción o se revierten en caso de error.
- NCF Secuencial: SecuenciaNCFService gestiona de forma thread-safe la asignación de comprobantes fiscales, evitando duplicados en entornos concurrentes.
- Cierre de Turno: TurnoCajaService valcula automáticamente ventas totales en efectivo y tarjeta, comparando con el monto declarado por el cajero y registrando la diferencia.

5 MODELO FÍSICO DE DATOS

El modelo físico de datos define la estructura concreta de la base de datos PostgreSQL que soporta el sistema MaxLi Manager. Cada tabla corresponde a una entidad del dominio definida en el modelo de clases UML. Se utilizan tipos de datos nativos de PostgreSQL, claves foráneas con restricciones de integridad referencial y convenciones de nomenclatura en snake_case.



5.1 Modelo Físico de Datos

Las tablas del sistema se organizan en los siguientes grupos funcionales:

Grupo 1: Seguridad y Acceso

Tabla	Columna	Tipo	Restricción	Descripción
rol	id_rol	BIGSERIAL	PK	Identificador único del rol
rol	nombre	VARCHAR(50)	NOT NULL, UNIQUE	Nombre del rol (ADMIN, CAJERO, ALMACENERO)
usuario	id_usuario	BIGSERIAL	PK	Identificador único del usuario

Tabla	Columna	Tipo	Restricción	Descripción
usuario	username	VARCHAR(100)	NOT NULL, UNIQUE	Nombre de usuario para login
usuario	email	VARCHAR(150)	NOT NULL, UNIQUE	Correo electrónico del usuario
usuario	password_hash	VARCHAR(255)	NOT NULL	Hash bcrypt de la contraseña
usuario	estado	VARCHAR(20)	NOT NULL, DEFAULT 'ACTIVO'	Estado del usuario (ACTIVO/INACTIVO)
usuario	id_rol	BIGINT	FK → rol(id_rol)	Rol asignado al usuario

Grupo 2: Gestión de Caja

Tabla	Columna	Tipo	Restricción	Descripción
caja	id_caja	VARCHAR(50)	PK	Identificador de la caja (ej: CAJA-01)
caja	nombre	VARCHAR(100)	NOT NULL	Nombre descriptivo de la caja
caja	estado	VARCHAR(20)	NOT NULL	Estado: ACTIVA / INACTIVA
turno_caja	id_turno_caja	BIGSERIAL	PK	Identificador del turno
turno_caja	id_caja	VARCHAR(50)	FK → caja(id_caja)	Caja a la que pertenece el turno
turno_caja	id_usuario	BIGINT	FK → usuario(id_usuario)	Cajero responsable del turno
turno_caja	fecha_apertura	TIMESTAMP	NOT NULL	Fecha y hora de apertura del turno
turno_caja	fecha_cierre	TIMESTAMP	NULLABLE	Fecha y hora de cierre del turno

Tabla	Columna	Tipo	Restricción	Descripción
turno_caja	monto_inicial	DECIMAL(15, 2)	NOT NULL	Monto en caja al abrir el turno
turno_caja	monto_cierre	DECIMAL(15, 2)	NULLABLE	Monto declarado al cierre
turno_caja	ventas_efectivo	BIGINT	DEFAULT 0	Total de ventas en efectivo (FK a Venta)
turno_caja	ventas_tarjetas	BIGINT	DEFAULT 0	Total de ventas con tarjeta (FK a Venta)
caja_chica	id_caja_chica	BIGSERIAL	PK	Identificador de la caja chica
caja_chica	nombre	VARCHAR(100)	NOT NULL	Nombre descriptivo
caja_chica	fondo_base	DECIMAL(15, 2)	NOT NULL	Fondo base asignado
caja_chica	balance_actual	DECIMAL(15, 2)	NOT NULL	Balance actual disponible
caja_chica	id_usuario	BIGINT	FK → usuario(id_usuario)	Responsable de la caja chica
movimiento_caja	id_movimiento	BIGSERIAL	PK	Identificador del movimiento
movimiento_caja	tipo_movimiento	VARCHAR(50)	NOT NULL	INGRESO / EGRESO
movimiento_caja	fecha_hora	TIMESTAMP	NOT NULL, DEFAULT NOW()	Fecha y hora del movimiento
movimiento_caja	monto	DECIMAL(15, 2)	NOT NULL	Monto del movimiento
movimiento_c	concepto	VARCHAR(255)	NOT NULL	Descripción

Tabla	Columna	Tipo	Restricción	Descripción
aja		55)		n del movimiento
movimiento_aja	id_control_ncf	BIGINT	FK → secuencia_ncf(id_control_ncf)	NCF asociado (opcional)

5.1.1 Grupo 3: Catálogo de Productos

Tabla	Columna	Tipo	Restricción	Descripción
categoria	id_categoria	BIGSERIAL	PK	Identificador de la categoría
categoria	nombre	VARCHAR(100)	NOT NULL, UNIQUE	Nombre de la categoría
marca	id_marca	BIGSERIAL	PK	Identificador de la marca
marca	nombre	VARCHAR(100)	NOT NULL	Nombre de la marca
producto	id_producto	BIGSERIAL	PK	Identificador único del producto
producto	codigo	VARCHAR(100)	NOT NULL, UNIQUE	Código de barras o SKU
producto	nombre	VARCHAR(200)	NOT NULL	Nombre del producto
producto	descripcion	TEXT	NULLABLE	Descripción detallada
producto	precio_venta	DECIMAL(15,2)	NOT NULL	Precio de venta al público
producto	costo	DECIMAL(15,2)	NOT NULL	Costo de adquisición
producto	estado	VARCHAR(20)	NOT NULL, DEFAULT 'ACTIVO'	Estado lógico (ACTIVO/INACTIVO)
producto	id_categoria	BIGINT	FK → categoria(id_categoria)	Categoría del producto
producto	id_marca	BIGINT	FK → marca(id_marca)	Marca del producto

Grupo 4: Inventario y Almacén

Tabla	Columna	Tipo	Restricción	Descripción
almacen	id_almacen	BIGSERIAL	PK	Identificador del almacén

Tabla	Columna	Tipo	Restricción	Descripción
almacen	descripcion	VARCHAR(200)	NOT NULL	Nombre/descripción del almacén
existencia	id_existencia	BIGSERIAL	PK	Identificador del registro de stock
existencia	id_producto	BIGINT	FK → producto(id_producto), UNIQUE	Producto al que corresponde el stock
existencia	id_almacen	BIGINT	FK → almacen(id_almacen)	Almacén donde se encuentra
existencia	cantidad_actual	INTEGER	NOT NULL, DEFAULT 0, CHECK >= 0	Cantidad actual en stock
existencia	cantidad_minima	INTEGER	NOT NULL, DEFAULT 0	Nivel mínimo (Stock Alarma)
movimiento	id_movimiento	BIGSERIAL	PK	Identificador del movimiento de inventario
movimiento	tipo	VARCHAR(50)	NOT NULL	ENTRADA / SALIDA / AJUSTE
movimiento	estado	VARCHAR(20)	NOT NULL	Estado del movimiento
movimiento	fecha	DATE	NOT NULL, DEFAULT NOW()	Fecha del movimiento
movimiento	motivo	TEXT	NULLABLE	Motivo o descripción
detalle_movimiento	id_detalle_movimiento	BIGSERIAL	PK	Identificador del detalle
detalle_movimiento	id_movimiento	BIGINT	FK → movimiento(id_movimiento)	Movimiento al que pertenece
detalle_movimiento	id_existencia	BIGINT	FK → existencia(id_existencia)	Existencia afectada
detalle_movimiento	cantidad_antes	INTEGER	NOT NULL	Stock antes del movimiento
detalle_movimiento	cantidad	INTEGER	NOT NULL	Cantidad movida

Tabla	Columna	Tipo	Restricción	Descripción
detalle_movimiento	cantidad_despu	INTEGER	NOT NULL	Stock después del movimiento

Grupo 5: Ventas y Clientes

Tabla	Columna	Tipo	Restricción	Descripción
cliente	id_cliente	BIGSERIAL	PK	Identificador único del cliente
cliente	nombre	VARCHAR(200)	NOT NULL	Nombre completo del cliente
cliente	cedula_rnc	VARCHAR(30)	NULLABLE	Cédula o RNC fiscal
cliente	telefono	VARCHAR(20)	NULLABLE	Número de teléfono
cliente	email	VARCHAR(150)	NULLABLE	Correo electrónico
cliente	direccion	TEXT	NULLABLE	Dirección física
venta	id_venta	BIGSERIAL	PK	Identificador de la venta
venta	id_factura_propio	VARCHAR(50)	NOT NULL, UNIQUE	Número interno de factura
venta	fecha	TIMESTAMP	NOT NULL, DEFAULT NOW()	Fecha y hora de la venta
venta	id_control_ncf	BIGINT	FK → secuencia_ncf(id_control_ncf)	NCF asignado a la factura
venta	numero_comprobante	VARCHAR(20)	NULLABLE	Número NCF impreso en factura
venta	impuesto	DECIMAL(15,2)	NOT NULL, DEFAULT 0	ITBIS calculado
venta	descuento	DECIMAL(15,2)	NOT NULL, DEFAULT 0	Descuento total aplicado
venta	total	DECIMAL(15,2)	NOT NULL	Total de la venta
venta	estado	VARCHAR(20)	NOT NULL, DEFAULT 'COMPLETADA'	Estado: COMPLETADA, ANULADA

Tabla	Columna	Tipo	Restricción	Descripción
venta	id_cliente	BIGINT	FK → cliente(id_cliente)	Cliente de la venta (nullable)
detalle_venta	id_detalle_venta	BIGSERIAL	PK	Identificador del detalle
detalle_venta	id_venta	BIGINT	FK → venta(id_venta)	Venta a la que pertenece
detalle_venta	id_producto	BIGINT	FK → producto(id_producto)	Producto vendido
detalle_venta	cantidad	INTEGER	NOT NULL, CHECK > 0	Unidades vendidas
detalle_venta	precio_venta	DECIMAL(15,2)	NOT NULL	Precio unitario al momento de venta
detalle_venta	descuento_unidad	DECIMAL(15,2)	NOT NULL, DEFAULT 0	Descuento por unidad
detalle_venta	subtotal	DECIMAL(15,2)	NOT NULL	Subtotal de la línea
ingreso	id_ingreso	BIGSERIAL	PK	Identificador del ingreso
ingreso	id_cliente	BIGINT	FK → cliente(id_cliente)	Cliente que realiza el pago
ingreso	monto	DECIMAL(15,2)	NOT NULL	Monto ingresado
ingreso	tipo_ingreso	VARCHAR(50)	NOT NULL	Tipo: EFECTIVO, TARJETA, TRANSFERENCIA
ingreso	fecha	DATE	NOT NULL	Fecha del ingreso
ingreso	metodo	VARCHAR(50)	NOT NULL	Método de pago
ingreso	referencia	VARCHAR(100)	NULLABLE	Número de referencia del pago

Grupo 6: Compras y Proveedores

Tabla	Columna	Tipo	Restricción	Descripción
proveedor	id_proveedor	BIGSERIAL	PK	Identificador único del

Tabla	Columna	Tipo	Restricción	Descripción
				proveedor
proveedor	nombre_empresa	VARCHAR(200)	NOT NULL	Razón social del proveedor
proveedor	rnc	VARCHAR(30)	NOT NULL, UNIQUE	RNC fiscal del proveedor
proveedor	ubicacion	TEXT	NULLABLE	Dirección del proveedor
proveedor	vendedor	VARCHAR(150)	NULLABLE	Nombre del contacto vendedor
proveedor	telefono	VARCHAR(20)	NULLABLE	Teléfono de contacto
proveedor	email	VARCHAR(150)	NULLABLE	Correo de contacto
proveedor	estado	VARCHAR(20)	NOT NULL, DEFAULT 'ACTIVO'	Estado lógico (ACTIVO/INACTIVO)
orden_compra	id_orden_compra	BIGSERIAL	PK	Identificador de la orden de compra
orden_compra	id_proveedor	BIGINT	FK → proveedor(id_proveedor)	Proveedor de la orden
orden_compra	fecha	DATE	NOT NULL, DEFAULT NOW()	Fecha de emisión
orden_compra	total	DECIMAL(15,2)	NOT NULL	Total calculado de la orden
orden_compra	estado	VARCHAR(20)	NOT NULL, DEFAULT 'PENDIENTE'	PENDIENTE / RECEPCIONADO / ANULADO
detalle_orden_compra	id_detalle_orden_compra	BIGSERIAL	PK	Identificador del detalle
detalle_orden_compra	id_orden_compra	BIGINT	FK → orden_compra(id_orden_compra)	Orden a la que pertenece
detalle_orden_compra	id_producto	BIGINT	FK → producto(id_producto)	Producto ordenado
detalle_orden_compra	cantidad	INTEGER	NOT NULL, CHECK > 0	Unidades solicitadas

Tabla	Columna	Tipo	Restricción	Descripción
detalle_orden_compra	precio_unitario	DECIMAL(15,2)	NOT NULL	Precio pactado por unidad
detalle_orden_compra	subtotal	DECIMAL(15,2)	NOT NULL	Subtotal de la línea
nota_recepcion	id_nota_recepcion	BIGSERIAL	PK	Identificador de la nota de recepción
nota_recepcion	id_orden_compra	BIGINT	FK → orden_compra, UNIQUE	Orden de compra recepcionada
nota_recepcion	fecha	DATE	NOT NULL, DEFAULT NOW()	Fecha de recepción
nota_recepcion	estado	VARCHAR(20)	NOT NULL	RECEPCIONADO / PARCIAL
detalle_nota_recepcion	id_detalle_nota_recepcion	BIGSERIAL	PK	Identificador del detalle
detalle_nota_recepcion	id_nota_recepcion	BIGINT	FK → nota_recepcion	Nota a la que pertenece
detalle_nota_recepcion	id_detalle_orden_compra	BIGINT	FK → detalle_orden_compra	Detalle de orden referenciado
detalle_nota_recepcion	cantidad_recibida	INTEGER	NOT NULL, CHECK > 0	Cantidad físicamente recibida
detalle_nota_recepcion	observacion	TEXT	NULLABLE	Observaciones del almacenero
pago_proveedor	id_pago_proveedor	BIGSERIAL	PK	Identificador del pago
pago_proveedor	id_orden_compra	BIGINT	FK → orden_compra(id_orden_compra)	Orden pagada
pago_proveedor	monto_pagado	DECIMAL(15,2)	NOT NULL	Monto efectivamente pagado
pago_proveedor	fecha	DATE	NOT NULL	Fecha del pago
pago_proveedor	metodo	VARCHAR(50)	NOT NULL	Método: TRANSFERENCIA, CHEQUE, EFECTIVO

Tabla	Columna	Tipo	Restricción	Descripción
pago_proveedor	numero_referencia	VARCHAR(100)	NULLABLE	Referencia del pago

Grupo 7: Devoluciones y Notas de Crédito

Tabla	Columna	Tipo	Restricción	Descripción
devolucion	id_devolucion	BIGSERIAL	PK	Identificador de la devolución
devolucion	id_venta	BIGINT	FK → venta(id_venta)	Venta que origina la devolución
devolucion	id_control_ncf	BIGINT	FK → secuencia_ncf	NCF de la nota de crédito
devolucion	monto	DECIMAL(15,2)	NOT NULL	Monto total devuelto
devolucion	fecha	DATE	NOT NULL, DEFAULT NOW()	Fecha de la devolución
devolucion	motivo	TEXT	NOT NULL	Motivo de la devolución
devolucion	total	DECIMAL(15,2)	NOT NULL	Total calculado de la devolución
detalle_devolucion	id_detalle_devolucion	BIGSERIAL	PK	Identificador del detalle
detalle_devolucion	id_devolucion	BIGINT	FK → devolucion(id_devolucion)	Devolución a la que pertenece

Tabla	Columna	Tipo	Restricción	Descripción
detalle_devolucion	id_detalle_venta	BIGINT	FK → detalle_venta(id_detalle_venta)	Línea de venta devuelta
detalle_devolucion	cantidad	INTEGER	NOT NULL, CHECK > 0	Unidades devueltas
detalle_devolucion	subtotal	DECIMAL(15,2)	NOT NULL	Subtotal de la devolución
nota_credito	id_nota_credito	BIGSERIAL	PK	Identificador de la nota de crédito
nota_credito	id_venta	BIGINT	FK → venta(id_venta)	Venta original
nota_credito	id_devolucion	BIGINT	FK → devolucion(id_devolucion)	Devolución asociada
nota_credito	id_control_ncf	BIGINT	FK → secuencia_ncf	Secuencia NCF de la nota
nota_credito	fecha	DATE	NOT NULL	Fecha de emisión
nota_credito	motivo	TEXT	NOT NULL	Motivo de la nota de crédito
nota_credito	monto	DECIMAL(15,2)	NOT NULL	Monto acreditado
nota_credito	estado	VARCHAR(20)	NOT NULL, DEFAULT 'ACTIVA'	Estado: ACTIVA / APLICADA

Grupo 8: Ofertas y Comprobantes Fiscales

Tabla	Columna	Tipo	Restricción	Descripción
oferta	id_oferta	BIGSERIAL	PK	Identificador de la oferta
oferta	nombre	VARCHAR(150)	NOT NULL	Nombre de la promoción
oferta	fecha_inicio	DATE	NOT NULL	Inicio de vigencia
oferta	fecha_fin	DATE	NOT NULL	Fin de vigencia
oferta	activa	BOOLEAN	NOT NULL, DEFAULT TRUE	Si la oferta está activa
oferta_cantidad	id	BIGSERIAL	PK	Identificador
oferta_cantidad	id_oferta	BIGINT	FK → oferta(id_oferta)	Oferta a la que pertenece
oferta_cantidad	cantidad_requerida	INTEGER	NOT NULL	Cantidad mínima para activar
oferta_cantidad	cantidad_cobrada	INTEGER	NOT NULL	Cantidad efectivamente cobrada
oferta_descuento	id	BIGSERIAL	PK	Identificador
oferta_descuento	id_oferta	BIGINT	FK → oferta(id_oferta)	Oferta a la que pertenece
oferta_descuento	porcentaje	DECIMAL(5,2)	NOT NULL	Porcentaje de descuento (0-100)
secuencia_ncf	id_control_ncf	BIGSERIAL	PK	Identificador del control NCF
secuencia_ncf	nombre_tipo	VARCHAR(100)	NOT NULL	Tipo de comprobante (B01, B04, etc.)
secuencia_ncf	prefijo	VARCHAR(10)	NOT NULL	Prefijo del NCF
secuencia_ncf	secuencia_inicial	INTEGER	NOT NULL	Número inicial de la secuencia
secuencia_ncf	secuencia_actual	INTEGER	NOT NULL	Número actual en uso

Tabla	Columna	Tipo	Restricción	Descripción
secuencia_ncf	secuencia_fin	INTEGER	NOT NULL	Número máximo permitido
secuencia_ncf	fecha_vencimiento	DATE	NOT NULL	Fecha de vencimiento de la autorización

Índices Recomendados

Tabla	Columna(s)	Tipo de Índice	Justificación
producto	codigo	UNIQUE INDEX	Búsqueda frecuente por código de barras en POS
existencia	id_producto	INDEX	Join frecuente en verificación de stock
venta	fecha, estado	COMPOSITE INDEX	Reportes por rango de fechas y estado
detalle_venta	id_venta	INDEX	Join frecuente al cargar detalle de ventas
movimiento	fecha	INDEX	Consultas de auditoría por rango de fechas
orden_compra	estado	INDEX	Filtrado frecuente de órdenes pendientes
secuencia_ncf	nombre_tipo	INDEX	Búsqueda rápida de secuencia NCF activa
usuario	username, estado	COMPOSITE INDEX	Login y validación de usuarios activos

6 DEFINICIÓN INTERFAZ DE USUARIO DEL SISTEMA

6.1 Especificación de la Interfaz de Pantalla y de Navegación

<Introduzca contenido y borre cuadro>

En este apartado se incluirá el diseño detallado del comportamiento de la interfaz de usuario a partir de la especificación de la misma, obtenida en el proceso de análisis y de acuerdo con el entorno tecnológico definido. Se incluirá la especificación de cada una de las pantallas del sistema, incluyendo eventos que la afectan, información que contiene y campos obligatorios. Toda esta información tiene que ser consistente en tipo y características con lo especificado en el modelo físico de datos.

6.2 Catálogo de Controles y Elementos de Diseño

<Introduzca contenido y borre cuadro>

En este apartado se incluirán todos los controles utilizados para la construcción de la interfaz gráfica (botones, campos de texto, mensajes de aviso...), especificando sus propiedades y los estilos que se utilizan. También se incluirá el diseño de los mensajes de error, mensajes de aviso o advertencia que genera

6.3 Definición de Informes

<Introduzca contenido y borre cuadro>

En esta sección se indicarán los distintos informes que se van a generar, vinculándolos a las pantallas anteriormente definidas, llegado al máximo grado de detalle.

7 DEFINICIÓN INTERFAZ DE SERVICIOS DEL SISTEMA

7.1 Definición de la Interfaz de Servicios del Sistema

<Cumplimente tabla y borre cuadro>

En este apartado se deberá incluir la definición de la interfaz de servicios del sistema (previamente descrita en el documento de Análisis del Sistema).

Se deberá especificar para cada método, los tipos y formatos de los parámetros de entrada y salida; en caso de utilizar SW, se indicará tanto el XSD de entrada como de salida. Además, deberá indicarse la codificación de los posibles errores.

<Método>		
Descripción		
Parámetro Entrada	Tipo	Descripción
Parámetro Salida	Tipo	Descripción
Error	Código	Descripción

7.2 Servicios Consumidos por el Sistema

<Cumplimente tabla y borre cuadro>

De la misma manera que se define la interfaz ofrecida por nuestro sistema, tenemos que definir los servicios que el sistema va consumir.

<Método>		
Descripción		
Parámetro Entrada	Tipo	Descripción

Parámetro Salida	Tipo	Descripción
Error	Código	Descripción

8 INFORMACIÓN SOBRE TRAZABILIDAD

Esta sección obligatoria debe contener el conjunto de matrices de trazabilidad que se considere oportunas para identificar las dependencias entre los diferentes elementos que aparecen en este documento, con respecto al contenido del documento de análisis/requisitos. Al menos, se deberán incluir, la siguiente matriz:

- Matriz de trazabilidad de **Modelo de Clases** frente al **Modelo Físico de Datos**. Se indicará la correspondencia entre las clases del diseño, y las entidades del modelo físico.

9 ANEXOS [Opcional]

<Introduzca contenido y borre cuadro>

Cualquier aspecto importante para el desarrollo del sistema, deberá anexarse. Especialmente, se expondrán los aspectos sobre la integración con otros sistemas, bien por integración funcional, bien por delegación.

9.1 Anexo A. *Glosario de Acrónimos y Abreviaturas*

<Introduzca contenido y borre cuadro>

Este anexo debe contener una lista ordenada alfabéticamente de los acrónimos y abreviaturas que aparezcan en el documento.

Para facilitar la reutilización entre proyectos, los acrónimos y abreviaturas comunes a la mayoría de los proyectos aparecerán en este glosario separados de los términos específicos del dominio del problema.